



Explaining Random Forest and XGBoost with Shallow Decision Trees by Co-clustering Feature Importance

Ruggero G. Pensa¹ · Anton Crombach^{2,3,4} · Sergio Peignier⁵ · Christophe Rigotti³

Received: 23 April 2025 / Revised: 29 September 2025 / Accepted: 4 November 2025
© The Author(s) 2025

Abstract

Transparency is a non-functional requirement of machine learning that promotes interpretable models and easily explainable outcomes. Unfortunately, interpretable classification models, such as linear, rule-based, and decision tree models, are superseded by more accurate but complex learning paradigms, such as deep neural networks and ensemble methods. More specifically, for tabular data classification, models based on tree ensembles, such as random forest or XGBoost, are still competitive compared to deep learning ones and are often preferred to the latter. However, they share the same interpretability issues, due to the complexity of the learned model and, consequently, offer low explainability of predictions. Existing solutions consist of computing some feature importance score or extracting an approximate surrogate model from the learned tree ensemble. However, these methods lead to surrogate models with either poor fidelity or questionable comprehensibility. In this paper, we propose to improve this trade-off using Goodman-Kruskal's association measure to find groups of instances with predictions that are explained by shared groups of features. To build this structure, instances are first described by SHAP values, which capture local feature importance, and then co-clustered with features on the basis of these SHAP values. Next, a surrogate model is built as a set of shallow decision trees learned for the different groups of instances and subsets of relevant features. Our experiments show that our method produces surrogate models that explain random forest and XGBoost classifiers with competitive fidelity and higher comprehensibility compared to recent state-of-the-art competitors.

Keywords Explainable AI · Tree ensemble model · Co-clustering · SHAP value

Editors: Riccardo Guidotti, Anna Monreale, Dino Pedreschi.

Extended author information available on the last page of the article

1 Introduction

Thanks to the advances of deep learning methods, most recent contributions of the machine learning community target complex and multimedia data, where transformer-based architectures, such as large language models and vision transformer models show their potential. Yet, tabular data are crucial in many important and sensitive applications, such as credit scoring, loan granting, medical diagnosis, insurance premium definition, and so on. In such scenarios, “traditional” machine learning models, such as random forest (Breiman, 2001)—which is based on bagging—and XGBoost (Chen & Guestrin, 2016)—based on gradient-boosted decision trees (Friedman, 2001)—still outperform more sophisticated deep neural network architectures (McElfresh et al., 2023) (though attention-based methods show improved performance on tabular data (Arik & Pfister, 2021)). Unfortunately, the outcomes of random forest and XGBoost lack transparency in terms of both the classification model learned and the predictions made, as they are poorly interpretable and explainable. Indeed, the outstanding performance on tabular data are strongly linked to the ability of such methods to capture non-linear relationships among the features, which are intrinsically unintelligible for humans. This is a major problem in all those sensitive application areas where the prediction made by a machine has a direct and important impact on the life of human beings, as wrong or faulty decisions could harm their health, freedom, and dignity.

To address the challenge of transparent AI, the involved organizations and companies usually take two types of measures: either they abandon complex models in favor of more interpretable but less accurate ones (such as logistic regression or decision trees), or they turn towards interpretability approaches such as global surrogate models and local explanations. In particular, the second option provides a good trade-off between accuracy and transparency and is also encouraged by recent advances in the prosperous field of explainable AI (XAI) (Dwivedi et al., 2023). A rough classification of the main methods divides them into two categories: methods providing global interpretations of opaque or even black-box models (Guidotti et al., 2019; Saleem et al., 2022), and methods providing a local explanation on individual instances (Liang et al., 2021). Among the latter, the SHAP framework (Lundberg & Lee, 2017) has gained attention thanks to its strong theoretical foundation (Molnar, 2025). It is derived from a famous contribution to game theory made by Shapley (1953) and makes the SHAP framework particularly adapted to applications that must adhere to strong legal requirements, such as financial and medical ones.

Although SHAP values are very flexible and can be aggregated to provide global explanations, they are not free of criticism (Huang & Marques-Silva, 2024). For instance, they are less expressive than decision tree paths in explaining individual predictions. Decision paths, i.e., the sequence of decision tree nodes that are triggered during the prediction of the target variable for an input instance, can be expressed in the form of *if-then-else* rules on every feature participating in the prediction. Thus, compared to SHAP values, they provide more insight on the specific range of feature values that leads to one particular outcome. Consequently, they are well understood by humans and perfectly comply with legal transparency requirements. Of course, decision paths come with their own potential drawbacks. The surrogate decision trees computed on non-linear models can be extremely complex, leading to long decision paths that are less human-readable. Recent papers (Blanco-Justicia et al., 2020; Sagi & Rokach, 2020, 2021; Bonsignori et al., 2021) address this issue by building (sets of) decision trees from opaque or black-box classification models. Three of them

are specifically designed to transform a decision forest into a single interpretable decision tree by either directly merging them or exploiting a set of conjunction rules extracted from the original tree ensembles (Sagi & Rokach, 2020, 2021; Bonsignori et al., 2021). Those methods produce surrogates with high fidelity, but at the cost of deep (thus weakly comprehensible) decision trees.

To cope with the issues surrounding transparent AI, in this paper we propose a new interpretability approach to tree-based ensemble models for tabular data, which computes a surrogate model whose purpose is two-fold: (i) it provides a global interpretation of the original model in terms of a collection of shallow decision trees, each trained for a subset of the data space on a subset of features; (ii) it provides local explanations of predictions in terms of compact decision paths. To achieve the first goal, we start by computing SHAP values for the training data instances to assess the importance of each features in each corresponding prediction. Next, we compute a co-clustering of the matrix consisting of the training instances as its rows, of the features as its columns, and containing the corresponding SHAP values. This co-clustering optimizes an association measure, the Goodman-Kruskal's τ (Goodman & Kruskal, 1954), that expresses the strength of the link between a partition of the features and a partition of the data instances over the matrix. Then, for each cluster of instances, we train a decision tree on the subset consisting of the most important features for this set of instances. As a result, we obtain a surrogate model composed of a set of shallow decision trees, one per cluster of instances, each characterized by the distribution of the SHAP values for the features. For the second goal, i.e., providing local explanations, we compute SHAP values of the features for a single input instance of interest, and we map this instance to the closest cluster of training instances according to its SHAP value profile. Then, the tree learned on this cluster is used to compute a decision path for the input instance.

We report an extensive experimental validation, from which we conclude that our method achieves high fidelity scores and provides simpler explanations in terms of decision path length, compared to three recent state-of-the-art approaches for explaining random forest (Sagi & Rokach, 2020; Bonsignori et al., 2021) and XGBoost (Sagi & Rokach, 2021) models. For reproducibility, the source code of our framework and all scripts are available online at <https://github.com/rupensa/xccshap>.

This paper extends an earlier contribution (Pensa et al., 2024), which was presented at the Discovery Science 2024 conference, in several ways: (i) we provide a more in-depth presentation of the theoretical basis of the method and its intuition; (ii) we now consider not only XGBoost models but also random forest and extend the experimental section accordingly; (iii) we provide additional experiments showing that our method still provides accurate surrogate models when trained on relatively small samples of the training data instances.

2 Related Work

Since the advent of complex model learning, such as non-linear support vector machines, deep neural networks, and gradient boosting decision trees, a major criticism is their weak interpretability and the poor explainability of their outcomes. Despite their superior performances in most applications, in fact, the activation patterns triggered by a single prediction are far from being humanly comprehensible. This issue has fostered a new branch of research called explainable or interpretable machine learning (Molnar, 2025), which has

been the object of many surveys and review papers (Guidotti et al., 2019; Liang et al., 2021; Burkart & Huber, 2021; Milani et al., 2024).

Indeed, many recent research works have addressed the problems of providing global interpretations of opaque or even black-box models (Guidotti et al., 2019; Saleem et al., 2022), and explaining every decision made on individual input instances (Liang et al., 2021). Global interpretations are typically provided by measuring and visualizing the importance of features (or groups of them) in the overall model (Fisher et al., 2019; Greenwell & Boehmke, 2020; Inglis et al., 2022) or by computing an interpretable surrogate model (in general, a linear one or a decision tree) that tries to imitate the original one and to approximate its predictions (Craven & Shavlik, 1995; Guidotti et al., 2019; Herbringer et al., 2023). In the latter case, a surrogate model can be composed of several models for different subsets of the data space, as for instance to preserve information privacy (Blanco-Justicia et al., 2020).

Local explanations, instead, are supplied on individual predictions by computing local approximations (Ribeiro et al., 2016), anchors (Ribeiro et al., 2018), counterfactual explanations (Keane et al., 2021; Guidotti et al., 2019), or by measuring the contribution of each feature in the specific prediction (Strumbelj & Kononenko, 2010). Among the latter, the SHAP framework (Lundberg & Lee, 2017) in particular has gained attention thanks to its strong theoretical foundations (Molnar, 2025).

When considering the more specific field of tree-based ensemble model interpretability, an early work aims at exploiting the structure of a decision forest to iteratively select the most informative node and construct a single decision tree (Assche & Blockeel, 2007). Ensemble methods, such as the one introduced by Gulowaty and Wozniak (2021), have also been used for the interpretation of decision forests. An alternative approach, presented by Vandewiele et al. (2017) applies genetic algorithms to transform the forest into a single decision tree. As reported by Sagi and Rokach (2020), these methods cannot be applied on realistic models with a reasonably high number of trees, as the complexity of both methods is exponential with the size of the decision forest and the dimensionality of the data. Another method pursuing the same objective, but following a different strategy, is proposed by Bonsignori et al. (2021). Their approach combines the tree-merging procedure introduced by Fan and Li (2020) with a post-hoc explanation technique to generate a single, more interpretable decision tree. According to the authors, this tree should also be shallower, as the method allows for multi-way splits. However, as we demonstrate in our experiments, the algorithm incurs high computational costs, particularly when the number of trees and features is large.

Two recent papers (Sagi & Rokach, 2020, 2021) address the issue of model explainability respectively on random forest and XGBoost. Sagi and Rokach (2020) propose a method consisting of two main stages for reducing a decision forest to a single decision tree. In the first stage, a conjunction set representing the original decision forest is created. In the second stage, a decision tree that organizes the conjunction set in a tree structure is computed. The second approach (Sagi & Rokach, 2021) is tailored on XGBoost and is aimed at inferring an approximate decision tree from the learned model. To do that, a pruning strategy is initially performed on the pre-trained ensemble. Successively, a representative set of conjunctions is extracted from the pruned ensemble. Finally, a decision tree of a specified maximum depth is computed leveraging the conjunction set. Compared to the parameter-less method of Sagi and Rokach (2020), the last method requires the height of the tree to be specified as input parameter. These two approaches, together with the one by Bonsignori et al. (2021), are the

current references for tree-based ensemble model interpretation and will be used to assess our proposed method. As we will show in the experiments, all three methods end up with less comprehensible explanations.

3 Background on Local Feature Importance

In this proposal, the explanation method is based on a measure of the importance of each feature in each prediction. These scores are used to group instances and features to build simple decision trees that mimic the original model in different parts of the data space.

The search for ways to assess feature importance in predictions is a large and active research domain that has led to a variety of approaches. Several visual methods have been proposed so far; for instance, ICE (Goldstein et al., 2015) computes for one feature a collection of curves, each curve depicting for each instance the value of the model output when varying this feature value over its domain (other features kept unchanged). Inspecting such plots for different features helps the user gain insight on their effect on model output. Measuring the importance of features to quantify them is a slightly different problem, that also received a lot of attention. This resulted in two families of approaches to evaluate global feature importance in tree ensemble models. One is based on permuting between instances the values of the assessed feature, followed by measuring the effect on the model loss (Breiman, 2001). The other one considers in turn each feature and aggregates over the tree nodes the gains made when splitting on this feature (Hastie et al., 2009). While both aim at providing a global evaluation of the role of each feature in a model, they do not fit with our approach, which requires a local evaluation of the feature importance for each training data instance.

Local importance measures have been designed according to various objectives and constraints. Many of them are based on the local gradient of model output given an input, as the score proposed by Baehrens et al. (2010). This provides useful insight on the effect of a small variation of a feature value on the output. However, a feature can be very important to drive a prediction, while, due to some saturation effect, having a value for which a small local change barely affects the output. In this case, an evaluation based on the local gradient will not underline this feature as being important (Shrikumar et al., 2017). To handle this, a complementary view has been adopted, which consist of associating to each feature, for a given instance, a score that reflects the contribution of the feature to the model output when compared to a reference output level (e.g., the average model output over the training set). In this line of works, some approaches led to measures that are easily computed, as for instance the BREAKDOWN algorithm (Staniak & Biecek, 2018), but they lacked clear declarative semantics or did not establish properties showing their ability to assess features in a fair way. To capture feature contributions, well-established frameworks with a full declarative specification are the LIME (Ribeiro et al., 2016) and SHAP (Lundberg & Lee, 2017) proposals. The latter, SHAP, has been selected in this paper to measure feature importance since, when compared to LIME, it ensures that a number of fairness properties are satisfied.

Let us now define preliminary notations and briefly recall the SHAP framework (Lundberg & Lee, 2017), based on the computation of Shapley values for explaining individual classification outcomes (Strumbelj & Kononenko, 2014).

In the remainder of the paper, we will consider a dataset $D = \{d_1, \dots, d_n\}$ of n data instances represented in an m -dimensional feature space. The features are represented by their indices, $F = \{1, \dots, m\}$ being the set of all indices. We consider a classification task over a set of classes $\mathcal{C}$, performed by a model f associating a class $z = f(d)$ to an instance d .

We suppose that, for each class h of $\mathcal{C}$, the model can also output $f^h(d)$, i.e., the membership probability of d to class h .

The goal of the SHAP framework is to assign a local importance value (hereafter called SHAP value) to each feature of F in the prediction $z = f(d)$ for an instance d (Lundberg & Lee, 2017). According to Lundberg and Lee (2017) and Strumbelj and Kononenko (2014), and when adapted to a classification task, the SHAP value $\phi_v(f^h, d)$ measures the contribution of the feature v to the computation of probability $f^h(d)$.

Given E , a subset of the feature set F , then $f_E^h(d)$ denotes the value of $f^h(d)$ when only the values of features in E are known.¹ As a special case, $f_\emptyset^h(d)$ is the probability $f^h(d)$ when no knowledge about the features is available. The *local accuracy* property, of the SHAP values, ensures that

$$f^h(d) = f_\emptyset^h(d) + \sum_{v=1}^m \phi_v(f^h, d) \quad (1)$$

This property simply means that the sum of the contributions of the features, as captured by the SHAP values, must be equal to the output of the model when added to $f_\emptyset^h(d)$ (the default

model output). Note that each $\phi_v(f^h, d)$ can be either positive or negative, and according to this additive framework, the most important features are those with the largest $|\phi_v(f^h, d)|$.

The way SHAP values $\phi_v(f^h, d)$ are specified follows a seminal result in cooperative game theory (Shapley, 1953), which proposes to assign a fair reward to players by measuring their individual contributions (known as Shapley values) to a grand coalition F they participate in. If the players in coalition F are considered to be features, then, according to Lundberg and Lee (2017) and Strumbelj and Kononenko (2014), the SHAP values can be obtained as

$$\phi_v(f^h, d) = \sum_{E \subseteq F \setminus \{v\}} w(E) \left(f_{E \cup \{v\}}^h(d) - f_E^h(d) \right) \quad (2)$$

where w is a weight function that only depends on m (the overall number of features) and on $|E|$:

$$w(E) = \frac{|E|!(m - |E| - 1)!}{m!} \quad (3)$$

¹ In general, models cannot output directly $f_E^h(d)$, and most of the time it is approximated by averaging the model output when setting features in E according to d and varying the other features.

Informally, $\phi_v(f^h, d)$ can be seen as the weighted sum, over all subsets of features E , of the contribution of feature v , when v is added to the features in E to make the prediction.

Though computing SHAP values has a rather high computational cost in the general case, this complexity can be strongly reduced (from exponential to polynomial time) for tree-based ensemble models (Lundberg et al., 2020). For the models used in this paper, random forests and XGBoost models, enhanced and parallel algorithms exist that improve the computational speed. These are mainly based on TreeSHAP (Lundberg et al., 2020). Among them, GPUTreeSHAP (Mitchell et al., 2022) uses GPU computing facilities, FastTreeSHAP (Yang, 2021) adopts caching mechanisms, and Linear TreeSHAP (Yu et al., 2022) exploits properties of polynomials to reduce the execution time. In our experiments, we use FastTreeSHAP to compute the necessary SHAP values.

4 The XCCSHAP Framework

In this section, we introduce our framework providing compact and accurate explanations for classification models trained on tabular data through random forest or XGBoost. Before entering into the details of the framework, we first present a use case in which it could be deployed.

We consider a car insurance company that operates through the Web to propose its products to potential customers. To provide a quote of the insurance premium, the online application requires multiple pieces of information that feed one or multiple algorithms contributing to determining the applicant's risk category. A classification model is used to determine the risk class of the applicant, thus leading to the consequent premium. The algorithm has been previously trained on part of the historical tabular data owned by the company, which records both personal information (e.g., age, gender, job position) and risk factors (such as the number of previous accidents, yearly mileage).

Suppose now that a group of potential customers makes a complaint against the company, because of suspected discrimination based on supposed disparate treatment or impact for some protected category of people. If the machine learning model adopted is not explainable (as it is the case for gradient-boosted decision trees), to verify that the actual model is not biased and to be able to show its compliance with all laws and regulations in force, the company could resort to some global explanation model and look for some critical decision paths. Such a surrogate can also be computed before a new model is deployed in order to check its fairness, to search for other “strange” patterns, and even to verify the conditions triggering inaccurate or wrong predictions.

As we argued in Sect. 1, feature importance alone could not provide sufficient insight into the predictions, as it does not consider the range of values at the origin of the prediction. Decision paths, instead, are more effective because not only do they take into account the feature values at each decision node, but they are also more human-readable, as they can be represented as sequences of “if-then-else” rules. Unfortunately, decision trees tend to become rather large and hence less interpretable, when they reproduce complex decision patterns.

To address this issue, we propose leveraging SHAP values to derive a collection of shallow decision trees, each trained on a subset of instances and features. Our framework, called XCCSHAP (eXplanations through Co-Clustering of SHAP values), is described in detail

below. In Sect. 4.1, we define the relevant partitions of the instance and feature sets and explain how these are used to construct the surrogate model. The partitioning process itself is described in Sect. 4.2, while Sect. 4.3 illustrates how explanations for individual predictions are generated.

4.1 Cluster-Based Surrogate Model

The key intuition of the framework is to partition the training data in groups, such that the instances in a group share a set of features having a strong impact on the model output. Then, for each group of instances, a shallow decision tree based on a limited set of features can be built to mimic the prediction of the original model. The collection of these trees forms a surrogate model that gives a global interpretation of the original model. Additionally, each tree can provide, for its corresponding instances, local explanations through its compact decision paths.

The whole process is split into the following steps. First, it scores the features for each instance; second, it partitions the data in groups; and finally, for each group it projects the data on a subset of the complete feature set and builds a decision tree. When selecting a feature subset to build a decision tree for a group of instances, a feature is deemed non-important if, for all instances in the group, it contributes only weakly (relative to the other features) to the class probabilities in the original model. Such features are filtered out. Conversely, a feature that consistently provides a strong (positive or negative) contribution to the probability of at least one class across all instances in the group is considered potentially useful. This does not imply that this feature must appear in the decision path of each instance of the group, but it suggests that some pertinent splits in the decision tree for that group could be based on this feature. So, the feature is retained to describe the instances and the choice of using it or not in some decision paths is left to the tree learning method itself.

Starting with feature scoring, the steps of XCCSHAP are defined as follows. Let f be a classification model over a set of classes $\mathcal{C}$, trained using random forest or XGBoost on a set of training data instances $D = \{d_1, \dots, d_n\}$, with feature indices $F = \{1, \dots, m\}$. Let

$d_u \in D$ be the u -th data instance, we consider the SHAP values $\phi_v(f^h, d_u)$ representing the importance of the contribution of feature v to the probability of class h for the instance d_u . To capture the importance of v in computing all class probabilities for d_u , we choose to retain its highest contribution, that is its maximum over all classes. This importance score is called the maximum absolute SHAP value (MASHAP) of v for d_u , and is defined as:

$$\text{MASHAP}(v, d_u) = \max_{h \in \mathcal{C}} |\phi_v(f^h, d_u)| \quad (4)$$

Then, a normalized MASHAP (NMASHAP) is obtained by simply normalizing over the features for instance d_u :

$$\text{NMASHAP}(v, d_u) = \frac{\text{MASHAP}(v, d_u)}{\max_{v' \in F} \text{MASHAP}(v', d_u)} \quad (5)$$

All feature contributions for d_u are thus scaled² to values between 0 and 1. For an instance d_u , features having a NMASHAP of 1 are the features that, according to their SHAP values, contribute the most to the class probabilities computed by the model for d_u . On the contrary, features with a NMASHAP close to 0 are likely to have nearly no importance in the output made for d_u . This normalization is used to put all importance scores on the same scale, so as to make them suitable for comparison.

Let S be the non-negative matrix containing the NMASHAP values for every training data instance d_u and every feature v . The elements of S are noted s_{uv} , with

$s_{uv} = \text{NMASHAP}(v, d_u)$. Consider a partition $R = \{R_1, \dots, R_k\}$ of the row indices of S and a partition $C = \{C_1, \dots, C_l\}$ of the column indices of S . The partition R (resp. C) is a way to group the instances (resp. the features) in S . The key step of the method is to compute partitions R and C such that: (1) all instances in the same R_i tend to have a model output strongly explained by features in the same collection of groups of columns (i.e. same subset of C); and (2) all features in the same C_j tend to be important to explain the output for the instances in a particular collection of groups of rows (i.e., a same subset of R). For such partitions R and C , this means that the model output of most instances of a group R_i is likely to be explained by the same set of features, and thus it makes sense to try to build a model specific to this set of instances R_i . Moreover, this means also that a group C_j in C either contains features playing an important role to compute the output for the instance in R_i or is composed of features of weaker importance. This connection between the partition R and C can be used to select the features to learn a model dedicated to R_i . The strategy is to take from each C_j the most important features, for two reasons that depend on the kind of features in C_j , corresponding to the two following cases:

1. C_j contains features that tend to be important for the instances in R_i . Selecting the best features among them is a way to retain original model behavior, while having a new model built on fewer attributes.
2. The features in C_j tend to be non-important for the instances in R_i . This means that they are likely to be useless for most instances in R_i , but selecting the most important of these weak features can still allow their use in the model if needed for a few instances, and thus can improve fidelity to the original model.

The precise construction of such models (specific to the groups in R) in order to obtain a global surrogate model will be defined below and the computation of the partitions R and C will be detailed in the next section.

The surrogate model is based on a collection $\mathcal{DT} = \{DT_1, \dots, DT_k\}$ of k decision trees, each tree DT_i being learned for each one of the row clusters $R_i \in R$. For a cluster R_i , we select a dataset $D^i \subset D$:

$$D^i = \{d_u \in D \text{ s.t. } u \in R_i\} \quad (6)$$

²The degenerate case where d_u is such that $\phi_v(f^h, d_u) = 0$ for all classes and all features is excluded, ensuring that $\max_{v' \in F} \text{MASHAP}(v', d_u) \neq 0$.

and build a vector Z^i containing the target values of the instances in D^i . For each $d \in D^i$, this target value is simply $f(d)$, i.e., the value predicted by the model we want to explain. A tree DT_i is learned using instances in D^i , target values Z^i and a set of features F^i defined hereafter. As in the work of Sagi and Rokach (2021), the maximum depth of the tree is the only hyperparameter used.

Let us first define matrices M and $\overline{M}$, of dimension $k \times l$, that aggregate the values in S in blocks over the groups in R and C . The elements m_{ij} of M are the sums of the elements

in S for each pair $\langle R_i, C_j \rangle$:

$$m_{ij} = \sum_{u \in R_i} \sum_{v \in C_j} s_{uv} \quad (7)$$

The elements $\overline{m}_{ij}$ being simply the corresponding average for each pair $\langle R_i, C_j \rangle$:

$$\overline{m}_{ij} = \frac{m_{ij}}{|R_i| \cdot |C_j|} \quad (8)$$

Then, we select a subset F^i of the features F as follows:

$$F^i = \left\{ v \in F \text{ s.t. } \frac{\sum_{u \in R_i} s_{uv}}{|R_i|} \geq \overline{m}_{ij_v} \wedge \sum_{u \in R_i} s_{uv} > 0 \right\} \quad (9)$$

where j_v is the index of the group in C containing feature v . The first condition means that the average NMASHAP of v over R_i is not less than this average for all features of the same cluster (i.e., $\overline{m}_{ij_v}$). The second condition is needed to avoid retaining feature v when NMASHAP values of v for all instances in R_i are equal to zero.

Intuitively, a decision tree DT_i , learnt on the instances in R_i , is built using only the most important features of each group in C with respect to the predictions made by original model f for these instances in R_i . The whole procedure described above is used to construct the collection $\mathcal{DT}$ forming the surrogate model, and is formally presented in Algorithm 1.

4.2 Partitioning Instances and Features

As introduced in the previous section, the partitions R and C should be connected, so that each group of instances tends to be mainly explained by the features of the same groups in C , and each group of features tends to contribute to a large extent to the explanation of the instances of the same groups in R . Connected partitions can be made using co-clustering, a machine learning task that given an input matrix computes a partition on the rows and a partition on the columns simultaneously (Dhillon, 2001). This is done by optimizing an objective function that takes into account both partitions. The way the overall optimization algorithm searches for the optimal co-clustering can be roughly viewed as iteratively and alternately executing clustering on rows while taking advantage of dimensionality reduction on columns and vice-versa.

There exist several co-clustering algorithms based on different approaches, different optimization criteria and requiring, or not, the specification of the number of clusters (Biernacki et al., 2023). In our context, let $\text{sum}(M)$ be the sum of the elements of M , i.e., the sum of all importance scores for the instances and the features in S . Then, let us consider the way this sum is distributed over the groups of instances in R and over the groups of features in C . We do not want this distribution to be uniform, on the contrary, for each given set of instance R_i we want that the corresponding importance score is concentrated in a few of the C_j . And we want the same from the perspective of the groups in C , that is the importance for each group of features C_j being concentrated in a few of the R_i groups. For a matrix of positive elements like M , such connection between groups of rows and groups of columns can be measured by the τ coefficient of Goodman and Kruskal (1954) on matrix $\frac{1}{\text{sum}(M)}M$. There exist a few algorithms that build co-clusters by optimizing this τ (e.g., the one by Robardet and Feschet (2001)). Among them, thanks to a non-normalized τ version, the algorithm PB- τ CC (Battaglia et al., 2024) can determine the optimal number of clusters for rows and the one for columns, while avoiding the degenerate cases where nearly all elements are grouped together in one or two very large clusters. For this reason, we adopt this co-clustering algorithm in the XCCSHAP framework. Then, k and l , respectively the number of clusters in partition R and C , do not need to be provided, but are determined during co-clustering. As most co-clustering algorithms, the results depends on the initialization. To address this issue, a practical solution is to perform the co-clustering multiple times and select the best solution with respect to the measure that is optimized. This is the solution adopted in our experiments, where the number of restarts is set to 30. Here, the computational cost is not an issue, as PB- τ CC scales well (Battaglia et al., 2024).

Input: A dataset D with set of feature indices F , a classification model f

Result: A matrix S of NMASHAP values, a partitioning R (resp. C) of the rows (resp. columns) of S , a collection $\mathcal{DT}$ of decision trees

```

1 // Compute the NMASHAP matrix
2 foreach  $d_u \in D$  do
3   foreach  $v \in F$  do
4     MASHAP( $v, d_u$ )  $\leftarrow \max_{h \in C} |\phi_v(f^h, d_u)|$ ;
5   end foreach
6   foreach  $v \in F$  do
7      $s_{uv} \leftarrow \frac{\text{MASHAP}(v, d_u)}{\max_{v' \in F} \text{MASHAP}(v', d_u)}$ ;
8   end foreach
9 end foreach
10  $(R, C) \leftarrow \text{compute-instance-and-feature-partitions}(S)$ ;
11  $\mathcal{DT} \leftarrow \{\emptyset\}$ ;
12 // Build datasets  $D_i$  using  $(R, C)$ 
13 foreach  $R_i \in R$  do
14    $D^i = \{d_u \in D \text{ s.t. } u \in R_i\}$ ;
15    $F^i = \left\{ v \in F \text{ s.t. } \frac{\sum_{u \in R_i} s_{uv}}{|R_i|} > \bar{m}_{ij_v} \wedge \sum_{u \in R_i} s_{uv} > 0 \right\}$  // see Eq. (9)
16    $Z^i \leftarrow f(D^i)$ ; // Vector of classes predicted for instances in  $D^i$ 
17    $DT_i \leftarrow$  decision tree trained on instances  $D^i$ , features  $F^i$ , target values  $Z^i$ ;
18    $\mathcal{DT} \leftarrow \mathcal{DT} \cup \{DT_i\}$ ;
19 end foreach
20 return  $S, R, C, \mathcal{DT}$ 

```

Algorithm 1 XCCSHAP(D, F, f)

4.3 Explaining Individual Predictions Using XCCSHAP

Although our framework provides a global interpretation of the model as a set of decision trees, it can also be used to provide a human-readable explanation of individual predictions (i.e., a local explanation). In fact, once the surrogate model has been computed, one can exploit it to explain individual predictions made by model f for an instance d_t . This is done simply by associating d_t to a cluster R_i . Then, visualizing statistics about the NMASHAP values over cluster R_i provides a first level of explanation, for instance through a boxplot of SHAP values for each feature. Of course, XCCSHAP goes beyond such feature scoring by providing the decision path computed for d_t in the related decision tree DT_i . Associating R_i to d_t is trivial when d_t belongs to the set of instances D used to compute R , but for any unseen data instance $d_t \notin D$ its cluster assignment must first be determined. In this case, a proximity measure is required to find the best R_i for d_t and this measure must be consistent with the grouping criterion of the co-clustering algorithm. Thus, XCCSHAP uses the similarity measure of Battaglia et al. (2024), on which the comparison between data instances and cluster prototypes is based during the PB- τ CC co-clustering process. We refer the reader to the work of Battaglia et al. (2024) for the formal derivation of this measure. To make the presentation of XCCSHAP self-contained, below we recall the definition of the measure and sketch its informal interpretation.

In our context, this similarity can be expressed as follows. Each R_i is described by a normalized vector profile π_{R_i} over the C_j , defined by:

$$\pi_{R_i} = \langle \pi_{R_i}^1, \dots, \pi_{R_i}^j, \dots, \pi_{R_i}^l \rangle \text{ with } \pi_{R_i}^j = \frac{m_{ij}}{\sum_{j'=1}^l m_{ij'}} \quad (10)$$

Accordingly, the instance d_t is represented by π_{d_t} , its normalized vector profile of NMASHAP values aggregated over the C_j . This vector is defined by:

$$\pi_{d_t} = \langle \pi_{d_t}^1, \dots, \pi_{d_t}^j, \dots, \pi_{d_t}^l \rangle \text{ with } \pi_{d_t}^j = \frac{\sum_{v \in C_j} \text{NMASHAP}(v, d_t)}{\sum_{v \in F} \text{NMASHAP}(v, d_t)} \quad (11)$$

Let $\gamma(d_t, R_i)$ be the following weighted dot product of π_{d_t} and π_{R_i} :

$$\gamma(d_t, R_i) = \sum_{j=1}^l w_j \cdot \pi_{d_t}^j \cdot \pi_{R_i}^j \quad (12)$$

where w_j is a weight, depending on the sum of the importance of the features in C_j , and defined as $w_j = \frac{\text{sum}(M)}{\sum_{i=1}^k m_{ij}}$. The value of γ is positive and increases when π_{d_t} and π_{R_i} align better, with lower importance given to components having small weights.³ Then, for a given d_t , the similarity criterion of Battaglia et al. (2024) can easily be reformulated as the following preference score:

³ Intuitively the weight of a component j is increased when the group of columns C_j is sparsely populated, still maintaining then the influence of C_j on γ .

$$\text{pref}_{d_t}(R_i) = \left(\sum_{j=1}^l m_{ij} \right) \cdot (\gamma(d_t, R_i) - 1) \quad (13)$$

Since the elements m_{ij} are positive, this means that for poor alignments of π_{d_t} and π_{R_i} (i.e., $0 \leq \gamma(d_t, R_i) < 1$) the preference score is negative, and is penalized even more when R_i gather a large sub-total of elements in M (i.e., large $\sum_{j=1}^l m_{ij}$). On the contrary, these important R_i tend to increase the preference score of better alignments (i.e., when $\gamma(d_t, R_i) \geq 1$).

Using this preference score, d_t can be assigned the cluster index $i^t \in \{1, \dots, k\}$ that satisfies the following condition:

$$i^t = \arg \max_{i=1, \dots, k} (\text{pref}_{d_t}(R_i)) \quad (14)$$

Finally, to explain the prediction made by f on d_t , one can use the decision tree $DT_{i^t} \in \mathcal{DT}$ to compute the decision path as the sequence of split nodes traversed by d_t . However, one must be aware that the outcome of the prediction of $f(d_t)$ may be different from the one of $DT_{i^t}(d_t)$. Hence, our local explanation only makes sense when $DT_{i^t}(d_t) = f(d_t)$. If it is not the case, the method returns by default the raw SHAP values themselves as local explanation.

5 Experiments

In this section we present and discuss the results of our experiments, showing the effectiveness of XCCSHAP in providing a good level of explainability in terms of fidelity with respect to the original machine learning model and compactness of the explanations themselves. On a large number of datasets, we assess the behavior of XCCSHAP for a classification task. We apply our method to two different classifiers, both based on tree-based ensemble methods: gradient boosting trees and random forest. Regarding the former, we use XGBoost (Chen & Guestrin, 2016) as learning model with its implementation available online.⁴ For the latter, the random forest, we use the implementation provided by scikit-learn.⁵

The data are downloaded from the UCI Machine Learning Repository.⁶ Columns with more than 50% of missing values are removed, and after this step, rows with at least one missing value are filtered out. Categorical attributes are converted into numerical ones by applying one-hot encoding, due to the poor direct support for non-numeric attributes in most machine learning models. The statistics of the pre-processed datasets are given in Table 1.

Data are split into a training (70% of the data) and test set (the remaining 30%). When the number of training data instances is less than 1 500, we execute a grid search on the space of the hyper-parameters. Otherwise, we use the approach described by Li et al. (2017), that uses an adaptive resource allocation to focus on the most promising configurations. In both cases, to reduce biases we use a five-fold cross validation. Then we retrain the classifier on

⁴<https://xgboost.ai/>.

⁵<https://scikit-learn.org/stable/>.

⁶<https://archive.ics.uci.edu/datasets>.

Table 1 Description of the datasets used in our experiments. All values are computed after pre-processing

Dataset	Target var.	#class.	#rows	#feat.	F1(XGB)	F1(RF)
Adult	Income	4	47,621	108	0.3320	0.3361
AIDS	cid	2	2139	23	0.8337	0.8246
Bank marketing	y	2	30,907	44	0.7393	0.7159
Breast cancer	Class	2	277	39	0.5575	0.5758
Breast cancer W	Diagnosis	2	569	30	0.9623	0.9556
Car evaluation	Class	4	1728	21	0.9199	0.8902
Cong. voting rec	Class	2	232	32	0.9714	0.9714
Credit approval	A16	2	653	46	0.8400	0.8456
Glass identification	Type_of_glass	6	214	9	0.5928	0.7849
Glioma	Grade	2	839	26	0.8626	0.8792
HCV	Category	5	589	13	0.5216	0.5221
Heart disease	Num	5	297	13	0.2738	0.2505
Heart failure	Death_event	2	299	12	0.7644	0.7436
Image segmentation	class	7	210	19	0.8677	0.8921
Ionosphere	Class	2	351	34	0.9484	0.9590
Letter recognition	lettr	26	20,000	16	0.9364	0.9597
Students' Dropout	Target	3	4424	36	0.6854	0.6738
Spambase	Class	2	4601	57	0.9552	0.9559
SUPPORT2	hospdead	2	753	62	0.8127	0.8202
Wine	class	3	178	13	0.9666	0.9804
Yeast	localization_site	10	1484	8	0.5529	0.5485

Classification performance of XGBoost and random forest are reported in the two rightmost columns. Meaning of the abbreviations: *var.* variable, *clas.* classes, *feat.* features

the whole training set using the hyper-parameters found. The macro-averaged F1 score on the test set is reported in Table 1.

Once the model is trained, we apply XCCSHAP on the training set to compute the surrogate model and measure two performance indicators on the test set: the *fidelity* of the learned explanations and their *comprehensibility* (Molnar, 2025). To measure the fidelity in the classification task, we compute the macro-averaged F1 score of the target variables predicted by the surrogate model on the test set with respect to those predicted by XGBoost and random forest models. The other indicator, the comprehensibility, is measured by computing four values: the overall number of nodes and the maximum depth of the surrogate trees, and the average decision path length of the predictions on the test set. Additionally, we also compute the number of clusters of our explanation model (i.e., the number of shallow decision trees returned by XCCSHAP).

As competitors, we consider XGBTA (XGBoost Tree Approximator) by Sagi and Rokach (2021), SAME (single-tree approximation method) introduced by Bonsignori et al. (2021), and FBT (forest based tree) by Sagi and Rokach (2020). XGBTA approximates a forest of trees with a single decision tree built by combining pruning and conjunction set extraction (Sagi & Rokach, 2021); SAME merges forest trees by combining an intentional and extensional approach Bonsignori et al. (2021); finally, FBT is a parameter-less method that forms a new decision tree by hierarchically organizing a set of rule conjunctions representing the original decision forest (Sagi & Rokach, 2020). All the three meth-

ods are specifically designed to explain ensemble models trained with XGBoost (XGBTA) or random forest (SAME and FBT). Consequently, they are expected to outperform our method in terms of fidelity. However, we expect XCCSHAP to lead to less complex trees and shorter decision paths, because of the feature selection step performed by the algorithm. While FBT and SAME do not need any hyperparameter tuning, the maximum depth of the surrogate shallow decision trees is a hyperparameter for both XCCSHAP and XGBTA and it is determined by performing a grid search with 5-fold cross-validation in the set $\text{max_depth} = \{2, 3, 4, 5, 10, 100\}$. Since all competing methods generate a single decision tree, we adapt our approach to also produce one single tree, thus ensuring a fairer comparison. To this end, we merge all decision trees $DT_i \in \mathcal{DT}$ by introducing $k - 1$ additional decision nodes (where k is the number of clusters), whose sole purpose is to route the test instance d_t to the closest decision tree. Although a multi-way split at the root could be used, we opted for binary splits to maintain compatibility with the scikit-learn tree data structure, which only supports binary branching. In all comprehensibility evaluations (number of nodes, tree depth, and decision path length), these artificial nodes are included. In "Appendix A.2" a visual example of a XCCSHAP surrogate model merging the shallow decision trees $DT_i \in \mathcal{DT}$ is presented.

Table 2 Fidelity, in terms of macro-averaged F1 score, number of three nodes and maximum tree depth for explaining XGBoost models, using XCCSHAP and XGBTA

Dataset	Fidelity (F1-score)		# tree nodes		Tree depth	
	XCCSHAP	XGBTA	XCCSHAP	XGBTA	XCCSHAP	XGBTA
Adult	0.4704	-	373	-	29	-
AIDS	0.8757	0.8949	23	10,071	4	22
Bank marketing	0.8231	0.7822	147	1,323,623	10	45
Breast cancer	0.8618	0.6242	13	63	3	5
Breast cancer W	0.9620	0.9812	19	3461	4	22
Car evaluation	0.9570	0.9736	143	3241	13	21
Cong. voting rec	1.0000	1.0000	7	7	2	2
Credit approval	0.9181	0.9438	35	63	6	5
Glass identification	0.5951	0.6423	71	675	9	14
Glioma	0.9599	0.9720	33	1391	7	16
HCV	0.6848	0.5556	33	31	7	4
Heart disease	0.4282	0.5221	105	61	11	5
Heart failure	0.8822	0.8361	25	63	5	5
Image segmentation	0.9258	0.8818	25	655	6	14
Ionosphere	0.9896	0.9072	23	2803	6	22
Letter recognition	0.8612	-	4357	-	24	-
Students' dropout	0.8269	0.8373	67	157,879	7	34
Spambase	0.9542	0.9576	95	38,305	10	34
SUPPORT2	0.8942	0.5108	15	17,057	3	29
Wine	0.9667	0.9799	13	61	3	5
Yeast	0.5982	0.6141	465	4233	18	23
Weighted avg.	0.8402	0.8104	126.69	823985.54	18.80	37.19
# wins	9	11	18	2	16	3

The best results are highlighted in bold

The experiments were implemented in Python 3.8 and executed on a Linux server with 32 Intel Xeon Skylake cores at 2.1GHz and 256GB RAM. The data and the source code required to reproduce all the experiments are available online at <https://github.com/rupensa/xccshap>. Some additional experiments about the preference score (Eq. 13) and the stability of XCCSHAP are discussed in the Appendix.

5.1 Results with XGBoost and Random Forest

We report the results concerning XGBoost, including the fidelity, the number of tree nodes and the maximum tree depth for XCCSHAP and XGBTA in Table 2, while those concerning the experiments with random forest, using XCCSHAP and FBT are in Table 3. The number of clusters generated by XCCSHAP and the average decision path length for all algorithms are reported in Table 4. Running times are shown in Table 5. A timeout of 48 h has been set for all algorithms. Hence, when some results are missing, it is because this limit was exceeded. Since SAME could only terminate for few datasets (even when the timeout limit was extended to 72 h), we report the related results separately in Table 6. The total number of wins of all competing algorithms for all the performance indicators

Table 3 Fidelity, in terms of macro-averaged F1 score, number of three nodes and maximum tree depth for explaining random forest models, using XCCSHAP and FBT

Dataset	Fidelity (F1-score)		# tree nodes		Tree depth	
	XCCSHAP	FBT	XCCSHAP	FBT	XCCSHAP	FBT
Adult	0.4786	-	219	-	11	-
AIDS	0.8301	0.5812	21	7175	4	27
Bank Marketing	0.7473	0.4760	577	9151	13	37
Breast cancer	0.5974	0.5747	29	7115	6	28
Breast cancer W)	0.9745	0.9259	17	7831	4	21
Car evaluation	0.6458	0.6316	39	8011	6	20
Cong. voting rec	1.0000	1.0000	9	7569	3	17
Credit approval	0.9070	0.9439	49	7049	7	27
Glass identification	0.5719	0.5276	73	5941	7	17
Glioma	0.9721	0.9681	51	6965	7	22
HCV	0.5621	0.6279	33	5449	6	24
Heart disease	0.3079	0.2501	71	6247	10	23
Heart failure	0.7711	0.8892	29	7391	6	18
Image segmentation	0.8847	0.5138	21	6141	5	20
Ionosphere	0.9392	0.2694	31	8059	7	29
Letter recognition	0.8486	0.0689	4531	6141	24	30
Students' dropout	0.7485	-	143	-	7	-
Spambase	0.9324	0.7611	107	7993	8	47
SUPPORT2	0.7543	0.7319	57	6321	9	26
Wine	0.8752	0.9269	11	7759	3	18
Yeast	0.5476	0.1681	455	6391	19	22
Weighted avg	0.7907	0.3961	1657.22	7818.44	15.10	33.34
# wins	16	4	21	0	21	0

The best results are highlighted in bold

Table 4 Number of clusters identified by XCCSHAP and average path length for all competing algorithms

Dataset	# clust.		XGBoost		Random forest	
	XGB	RF	XCCSHAP	XGBTA	XCCSHAP	FBT
Adult	10	7	4.97±1.20	-	5.14±0.83	-
AIDS	2	2	3.40±0.49	13.88±2.82	2.86±0.87	9.10±1.65
Bank marketing	6	4	5.24±0.68	23.86±5.29	5.78±4.01	10.90±3.97
Breast cancer	2	5	2.99±0.11	5.00±0.00	2.79±0.58	9.46±2.67
Breast cancer W	3	2	2.90±0.30	11.88±3.59	2.87±0.56	12.49±3.17
Car evaluation	5	3	3.02±2.49	11.46±1.69	3.18±1.47	11.61±2.74
Cong. voting rec	2	2	2.00±0.00	2.00±0.00	2.21±0.41	9.91±1.68
Credit approval	3	6	3.85±0.98	5.00±0.00	2.89±1.14	9.85±2.36
Glass identification	3	4	4.71±1.05	8.72±2.11	4.17±0.85	11.94±1.86
Glioma	5	5	2.98±0.42	10.45±2.93	3.75±0.55	14.27±4.05
HCV data	3	2	3.36±0.75	4.00±0.00	3.19±1.17	5.81±2.22
Heart disease	3	3	5.53±1.93	4.99±0.10	5.38±1.95	10.84±2.16
Heart failure	4	4	2.76±0.43	5.00±0.00	2.79±0.74	11.73±1.67
Image segmentation	6	3	2.06±0.89	8.25±2.27	2.92±0.74	11.73±2.21
Ionosphere	4	3	2.92±0.96	11.71±3.64	3.09±0.81	10.94±1.88
Letter recognition	3	3	11.21±3.47	-	11.37±3.00	10.07±1.41
Students' dropout	5	3	3.80±0.64	17.72±4.42	5.46±0.50	-
Spambase	6	5	3.93±1.29	18.59±5.48	4.32±1.24	18.43±7.49
SUPPORT2	2	3	3.00±0.00	13.88±3.94	3.69±1.41	11.62±3.21
Wine	3	3	2.02±0.13	5.00±0.00	2.00±0.00	14.00±1.77
Yeast	6	7	6.77±2.94	12.71±3.06	6.71±3.30	8.21±2.67
Avg. path length/instance			5.89	19.99	6.20	11.09
# wins			19	1	19	1

The best results are highlighted in bold

are also reported. Note that ties are not considered in computing the number of wins. The overall running times (in seconds), correspond to the computation of the surrogate models. In the case of XCCSHAP, they encompass also the computation of the SHAP values, and include hyperparameter tuning for both XCCSHAP and XGBTA. For XCCSHAP, we also report the partial runtime, which does not account for time needed to compute the SHAP values.

Furthermore, we also show other aggregate measures, i.e., the average for the fidelity score, the number of tree nodes and the maximum tree depth, all weighted by the number of test instances of each dataset. Finally, we compute the average running time per instance and the average decision path length per instance.

In terms of fidelity, XCCSHAP and XGBTA achieve similar performance (see Table 2): none of the two methods clearly outperforms the other one, as confirmed by the slightly higher number of wins of XGBTA and the higher weighted average fidelity of XCCSHAP. However, the performance gap is clear when we compare the two methods with respect to comprehensibility in terms of tree nodes, tree depth (Table 2), and average path length (Table 4). The average depth of XGBTA is twice the one of XCCSHAP and, with few exceptions, XCCSHAP produces shallower and less complex trees. The average path length of the trees learned by XCCSHAP rarely exceeds 5, while

Table 5 Running time in seconds. Partial runtime for XCCSHAP does not take into account the time needed to compute the SHAP values

Dataset	XCCSHAP (XGB)		XGBTA	XCCSHAP (RF)		FBT
	Partial	Overall	Overall	Partial	Overall	Overall
Adult	604.76	613.93	-	1461.99	21,386.18	-
AIDS	13.34	13.36	772.05	15.01	24.52	6576.18
Bank marketing	517.13	517.86	33396.98	487.85	2707.67	54977.91
Breast cancer	1.50	1.50	74.82	3.41	3.72	1366.64
Breast cancer W	5.94	5.94	136.73	6.25	6.29	1265.22
Car evaluation	13.00	13.07	1133.47	14.57	28.84	4013.40
Cong. voting rec	0.44	0.45	61.44	1.26	1.30	880.82
Credit approval	5.08	5.09	247.60	23.97	24.41	1537.26
Glass identification	1.12	1.13	117.51	1.36	1.45	389.46
Glioma	3.81	3.81	165.81	9.40	10.02	824.65
HCV	2.19	2.20	147.75	3.57	3.80	1028.99
Heart disease	1.80	1.81	370.89	2.54	2.87	1877.38
Heart failure	2.39	2.40	79.59	3.29	3.31	376.93
Image segmentation	1.03	1.04	973.82	1.27	1.34	498.99
Ionosphere	2.24	2.25	104.03	3.31	3.38	1138.48
Letter recognition	250.47	278.53	-	192.62	2410.66	62691.20
Students' dropout	39.02	39.15	5724.83	50.61	145.44	-
Spambase	36.13	36.20	3235.91	41.66	67.66	4293.39
SUPPORT2	5.44	5.45	3536.25	8.98	9.54	1359.28
Wine	0.62	0.62	61.29	0.97	0.98	397.40
Yeast	7.74	7.85	17835.05	7.25	20.50	7899.85
Avg. time/instance	0.0104	0.0105	2.2390	0.0162	0.0609	3.8922
# wins		21	0		21	0

The best results are highlighted in bold

Table 6 Fidelity, surrogate model complexity measures and running time of SAME compared with those obtained by XCCSHAP

Perf. measure	Car evaluation		Glioma		Heart failure	
	XCCSHAP	SAME	XCCSHAP	SAME	XCCSHAP	SAME
Fidelity (F1)	0.6458	1.000	0.9721	0.9920	0.7711	0.9280
# tree nodes	39	808,303	51	71,033,474	29	2,477,154
Tree depth	6	21	7	29	6	25
Avg. path len	3.2±1.5	14.1±5.9	3.8±0.6	17.6±3.2	2.8±0.7	13.4±2.5
Runtime (s)	28.84	2303.12	10.02	74316.44	3.31	382.54

The best results are highlighted in bold

decision paths of the trees built by XGBTA are larger for almost all datasets and often more than twice as long as those of XCCSHAP. For time performance, the running times of XGBTA are at least one order of magnitude larger than those of XCCSHAP. On some datasets (e.g., Bank Marketing and Yeast) XGBTA takes hours of computation. On Adult and Letter Recognition, XGBTA failed in terminating within the 48 h' timeout limit.

Similar observations apply to the experiments with random forest, where XCCSHAP is compared to FBT. Note that the latter failed with Adult and Students' Dropout, because of the timeout limit. In terms of fidelity (Table 3), XCCSHAP outperformed FBT in 16 of 21 datasets and always led to much shallower trees with shorter decision paths (Table 4). In terms of running time, FBT is ways slower than XCCSHAP, for most datasets by at least two orders of magnitude (Table 5).

We wondered if the results concerning the fidelity were statistically significant. Since we do not know the underlying distributions, we applied a two-sided paired t-test, which assumes a normal distribution, and a Wilcoxon rank-sum test (also known as the Mann–Whitney U test (Mann & Whitney, 1947)), which does not make a normality assumption. Indeed, the latter is a non-parametric statistical test to verify the null hypothesis that two samples come from the same population, regardless of its distribution. The t-test confirmed that the difference in fidelity are statistically significant ($p < 0.05$). Instead, the null hypothesis of the Wilcoxon rank-sum test could not be rejected ($p > 0.1$).

Finally, we report the results obtained by SAME (see Table 6) on the three datasets for which it can terminate within 72 h (car evaluation, Glioma and heart failure). Indeed, SAME is able to find surrogate trees with high fidelity, when compared to XCCSHAP, but this comes at the cost of both higher tree complexity (in terms of tree nodes, tree depth and average decision path length) and running time. This could make its application more problematic in realistic scenarios.

The experiments reported in this section, show that XCCSHAP is competitive in terms of fidelity and better in terms of comprehensibility, when compared to XGBTA and FBT. SAME seems to better approximate RF trees, but we were able to assess its behavior only on three small datasets. As a general observation, the fidelity obtained by XGBTA, SAME, and FBT comes at the cost of more complex surrogate models to reproduce the predictions of the original model. Learning and using these surrogate models also has an impact on running time, which is greater for XGBTA, SAME, and FBT than XCCSHAP.

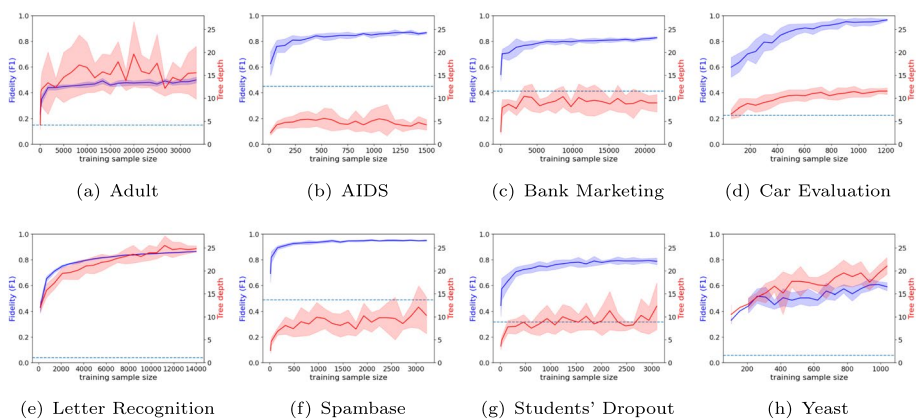


Fig. 1 F1 and tree depth depending on sample size for eight datasets. Data shown as mean $\pm$ standard deviation. The horizontal dashed lines represent the fidelity of the random classifier

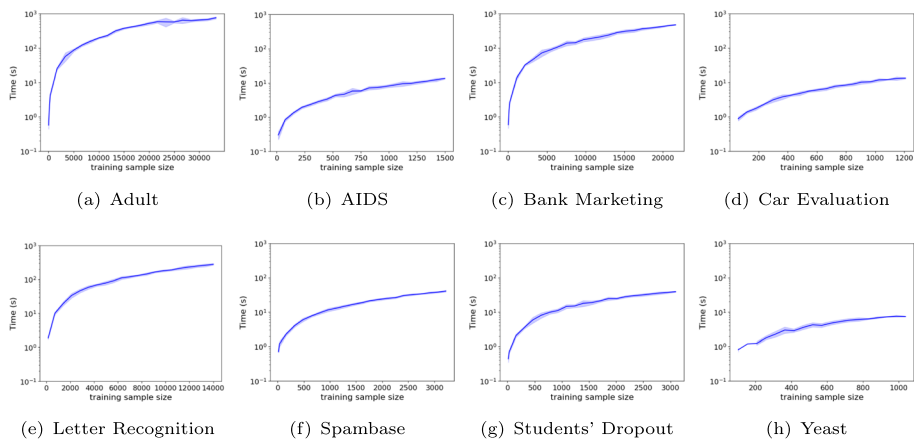


Fig. 2 Running time (in seconds, log scale) depending on sample size for eight datasets. Data shown as mean $\pm$ standard deviation

5.2 Performance of XCCSHAP when Using Samples of the Training Set

The experiments show that XCCSHAP leads to the shortest running time, but can sampling be used to reduce further the computational cost while still achieving reasonable fidelity performances? To test this hypothesis, we applied XCCSHAP on training data samples of increasing size, from 0.01% up to 100% of the training set. We report results on the eight largest datasets (adult, AIDS, bank marketing, car evaluation, letter recognition, spambase, students' dropout and yeast) for XGBoost models, by averaging over 10 different samples for each sample size. Fidelity and comprehension are shown in Fig. 1 and running time in Fig. 2. We observe that computational time can be significantly reduced without affecting fidelity too much. With small samples, around 5 to 10% of the training data, both the macro-averaged F1 score and the maximum tree depth are comparable to those obtained using the whole training set, while running times are reduced by roughly an order of magnitude. Noticeably, all fidelity scores are always above the F1-score of the random classifier (horizontal dashed line in all plots).

Beyond the reduction of the execution time, when the sample size is reduced, in some cases the surrogate trees are also shallower. This suggests that, for some datasets, sampling could have a positive effect on the trade-off between fidelity and comprehensibility.

6 Conclusion

We have presented XCCSHAP, an explanation method that provides surrogate models with competitive fidelity and very compact decision paths to interpret random forest and XGBoost predictions. Given a trained model and its corresponding training data, our method exploits SHAP values to extract a co-clustering of data instances and features. Next, it computes one shallow decision tree per cluster of instances using a subset of features. Each cluster is thus characterized by its SHAP value distribution and corresponding decision tree. Moreover, XCCSHAP is applicable beyond the training data, by assigning any new

data instance to the closest SHAP-based cluster. The prediction made for this new instance can then be explained by interrogating the corresponding shallow decision tree. Through extensive experiments on more than twenty real-world datasets, we show that our approach is competitive with three state-of-the-art methods in terms of fidelity. Moreover, it outperforms them in terms of comprehensibility, measured as the depth and number of nodes of the surrogate tree as well as the average length of the decision path of the explanations. Last but not least, our method is also orders of magnitude faster than the competing ones.

As future work, we prioritize three directions of research. First, a one-hot encoding scheme was used for the categorical attributes. Thus, each of such attributes leads to the creation of a set of binary attributes that are processed in isolation by XCCSHAP, while it could be more coherent to consider them as forming an indivisible group during feature selection. Such a strategy could be investigated to improve the generalization ability of the surrogate models and to improve their fidelity by also adopting, for instance, alternative encoding methods. Another direction of work follows up on the promising results of using only a small set of samples from the training set. A well-designed sampling method may not only reduce the execution time, but could also find other interesting properties of, and trade-offs between, fidelity and comprehensibility.

Finally, the cluster membership mechanism could be improved by combining it with a learnable post-hoc weighting function, thus enabling the model to leverage both the cluster membership strength and an optimized importance weight when selecting the appropriate shallow tree for explanation generation.

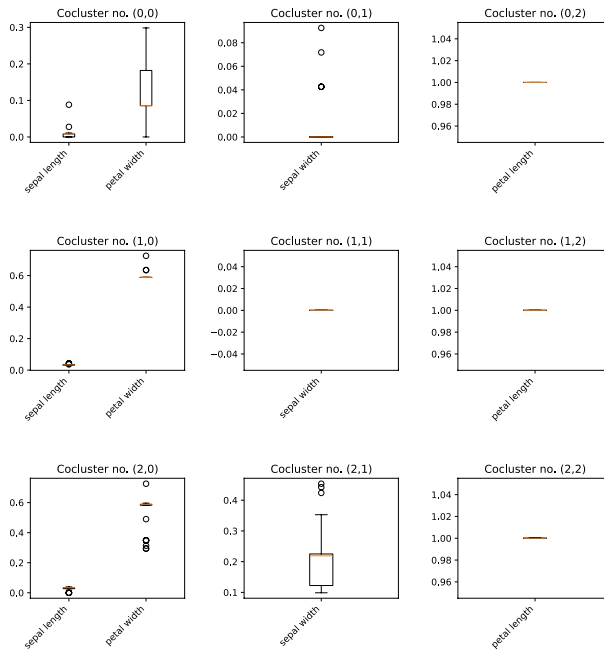
Appendix A. Additional Experiments

A.1. Stability of the Explaining Models

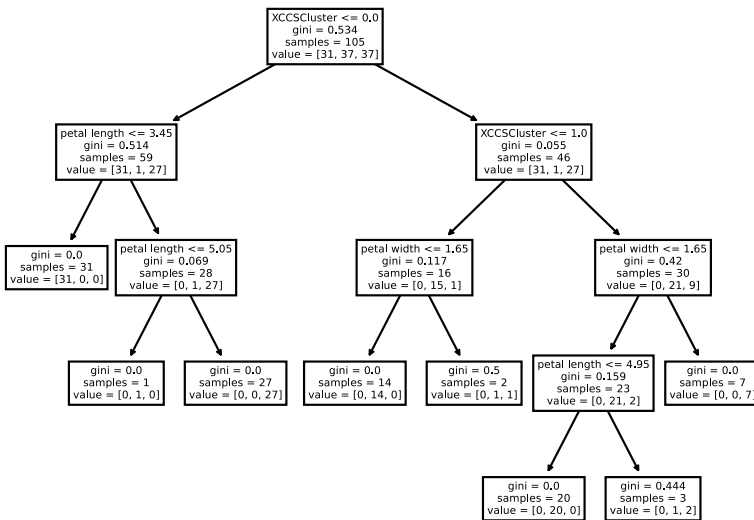
To assess the stability of XCCSHAP with respect to classifier initialization, we performed an additional experiment on the datasets where XGBoost achieved the lowest performance in our previous analyses. During the grid search phase of the classification task, we varied the random seed, obtaining ten distinct fine-tuned classifiers. For each of these models, we applied XCCSHAP and evaluated the stability of the surrogate trees in terms of fidelity score and maximum depth. We report both the average and the standard deviation of these measures. The results, shown in Table 7, indicate that XCCSHAP achieves rather stable fidelity score and maximum depth of the decision trees over the different applications of the method on a same dataset.

Table 7 Mean and standard deviation of the fidelity and maximum tree depth achieved by XCCSHAP on ten XGBoost models with different initialization for each dataset. Second column reports mean and standard deviation of the F1-score of the XGBoost classifier it-self

Dataset	Class. F1	Fidelity (F1-score)	Tree depth
Adult	0.3385±0.0038	0.4711±0.0261	11.1±2.0
Breast cancer	0.6175±0.0400	0.8112±0.1491	5.8±2.1
Glass identification	0.8321±0.1005	0.6202±0.0672	10.5±1.9
HCV	0.8157±0.1476	0.5500±0.1317	7.0±1.6
Heart disease	0.4665±0.1142	0.3817±0.0810	7.8±2.9
Students' dropout	0.7425±0.0279	0.7772±0.0287	9.6±3.2
Yeast	0.5963±0.0911	0.5209±0.0586	19.1±2.6



(a) NMASHAP co-clustering



(b) XCCSHAP Decision Tree

Fig. 3 Example of visualization of the surrogate model that explains XGBoost on Iris, which includes the boxplot of the NMASHAP values in three co-clusters (a) and the corresponding decision tree (b)

A.2. Visualization of the Surrogate Model

Figure 3 presents a visual example of the XCCSHAP explanatory model derived from the well-known Iris dataset. The visualization consists of a 3×3 grid of boxplots (Fig. 3a), which display the distribution of NMASHAP values within each co-cluster, alongside a decision tree (Fig. 3b) whose top-level nodes branch into three subtrees, one for each row cluster.

A.3. Behavior of the Preference Score

To investigate the relationship between the preference score and prediction quality, we conducted an additional experiment. Specifically, we measured the preference scores for two groups of instances: correctly predicted and incorrectly predicted. We then compared the mean preference score between the two groups to assess whether it is higher for correctly predicted instances (Condition #1 in Table 8). In addition, we applied two one-sided t-tests

Table 8 Behavior of the preference score (Eq. 13) w.r.t. the fidelity of individual predictions. Condition #1 is met when the average value of $pref_{dt}(R_i)$ is higher for the correctly predicted instances than for the incorrectly predicted ones. Condition #2 (resp. Condition #3) is satisfied when the mean preference score of correctly predicted instances is significantly greater (or, respectively, smaller) than that of incorrectly predicted instances, with statistical significance assessed at p -value < 0.05

Dataset	XGBoost			Random forest		
	Cond. #1	Cond. #2	Cond. #3	Cond. #1	Cond. #2	Cond. #3
Adult	✓	✓	-	✓	✓	-
AIDS	✓	✓	-	-	-	-
Bank marketing	✓	✓	-	✓	✓	-
Breast cancer	-	-	-	-	-	-
Breast cancer W	-	-	-	-	-	-
Car evaluation	-	-	✓	✓	-	-
Cong. voting rec	✓	✓	-	✓	✓	✓
Credit approval	✓	-	-	-	-	-
Glass identification	✓	-	-	✓	✓	-
Glioma	-	-	-	✓	✓	-
HCV data	-	-	-	-	-	✓
Heart disease	-	-	-	✓	✓	-
Heart failure	✓	-	-	-	-	✓
Image segmentation	✓	✓	-	✓	✓	-
Ionosphere	✓	✓	-	✓	✓	-
Letter recognition	✓	✓	-	✓	✓	-
Students' dropout	-	-	✓	✓	-	-
Spambase	✓	✓	-	✓	✓	-
SUPPORT2	✓	✓	-	✓	-	-
Wine	✓	-	-	✓	-	-
Yeast	-	-	-	✓	-	-
Counts	13	9	2	15	10	3

for independent samples to test whether the mean preference score of the correctly predicted group is significantly greater than that of the incorrectly predicted group (Condition #2 in Table 8), and vice versa (Condition #3 in Table 8), with statistical significance assessed at p -value < 0.05 . The results show that, in most datasets, higher preference scores are associated with better predictions, although the difference is not always statistically significant. Conversely, only in very few cases, the average preference score is significantly higher in the second group (of incorrectly predicted instances) than that of the first group. These observations hold for both XGBoost and random forest.

Acknowledgements R.G. Pensa is member of Gruppo Nazionale Calcolo Scientifico – Istituto Nazionale di Alta Matematica (GNCS-INdAM). C. Rigotti is a member of the LabEx IMU (ANR-10-LABX-0088) of Université de Lyon.

Author Contributions R.G.P., A.C., S.P. and C.R.: Conceptualization, Methodology, Investigation, Writing - original draft, Writing -review & editing. R.G.P.: Software, Validation.

Funding Open access funding provided by Università degli Studi di Torino within the CRUI-CARE Agreement. For this work, R.G. Pensa has received a grant from Inria under its visiting scholar program. This work was also partially funded by BQR INSA Lyon 2023 Neurinfo, ANR project C2R-IA ANR-22-CE56-0005, PEPR Santé Numérique project 22-PESN-0002, Fondation ARC grant ARCPJA22020060002212, and Institut National du Cancer grant PLBIO22-071.

Data Availability All data are available online and accessible to everyone.

Declarations

Conflict of interest R.G. Pensa is member of the Editorial Board. The authors have no further Conflict of interest to declare that are relevant to the content of this article.

Ethics Approval and Consent to Participate The authors declare that this research did not require Ethics approval or Consent to participate since it does not concern human participants or human or animal datasets.

Consent for Publication The authors of this manuscript consent to its publication.

Availability of Data and Material All data are available online and accessible to everyone.

Code Availability Source code and scripts used in our experiments are available at <https://github.com/rupe/rupe>.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

Arik, S. Ö., & Pfister, T. (2021). Tabnet: Attentive interpretable tabular learning. In *Proceedings of the AAAI/IAAI/AAAI 2021* (pp. 6679–6687).

- Assche, A. V., & Blockeel, H. (2007). Seeing the forest through the trees: Learning a comprehensible model from an ensemble. In *Proceedings of the ECML 2007* (pp. 418–429).
- Baehrens, D., Schroeter, T., Harmeling, S., Kawanabe, M., Hansen, K., & Müller, K.-R. (2010). How to explain individual classification decisions. *Journal of Machine Learning Research*, 11, 1803–1831.
- Battaglia, E., Peiretti, F., & Pensa, R. G. (2024). Fast parameterless prototype-based co-clustering. *Machine Learning*, 113(4), 2153–2181.
- Biernacki, C., Jacques, J., & Keribin, C. (2023). A survey on model-based co-clustering: High dimension and estimation challenges. *Journal of Classification*, 40(2), 332–381.
- Blanco-Justicia, A., Domingo-Ferrer, J., Martínez, S., & Sánchez, D. (2020). Machine learning explainability via microaggregation and shallow decision trees. *Knowledge-Based Systems*, 194, Article 105532.
- Bonsignori, V., Guidotti, R., & Monreale, A. (2021). Deriving a single interpretable model by merging tree-based classifiers. *Proceedings of DS 2021, Halifax, NS, Canada, October 11–13, 2021* (Vol. 12986, pp. 347–357). Springer.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- Burkart, N., & Huber, M. F. (2021). A survey on the explainability of supervised machine learning. *Journal of Artificial Intelligence Research*, 70, 245–317.
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the ACM SIGKDD 2016* (pp. 785–794).
- Craven, M. W., & Shavlik, J. W. (1995). Extracting tree-structured representations of trained networks. *Proceedings of NIPS 1995, Denver, CO, USA, November 27–30, 1995* (pp. 24–30). MIT Press.
- Dhillon, I. S. (2001). Co-clustering documents and words using bipartite spectral graph partitioning. In *Proceedings of the ACM SIGKDD 2001* (pp. 269–274).
- Dwivedi, R., Dave, D., Naik, H., Singhal, S., Rana, O. F., Patel, P., & Ranjan, R. (2023). Explainable AI (XAI): Core ideas, techniques, and solutions. *ACM Computing Surveys*, 55(9), 194:1–194:33.
- Fan, C., & Li, P. (2020). Classification acceleration via merging decision trees. In *Proceedings of FODS '20, virtual event, USA, October 19–20, 2020* (pp. 13–22). ACM.
- Fisher, A., Rudin, C., & Dominici, F. (2019). All models are wrong, but many are useful: Learning a variable's importance by studying an entire class of prediction models simultaneously. *Journal of Machine Learning Research*, 20, 177:1–177:81.
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 1189–1232.
- Goldstein, A., Kapelner, A., Bleich, J., & Pitkin, E. (2015). Peeking inside the black box: Visualizing statistical learning with plots of individual conditional expectation. *Journal of Computational and Graphical Statistics*, 24(1), 44–65.
- Goodman, L. A., & Kruskal, W. H. (1954). Measures of association for cross classification. *Journal of American Statistical Association*, 49, 732–764.
- Greenwell, B. M., & Boehmke, B. C. (2020). Variable importance plots: An introduction to the VIP package. *The R Journal*, 12(1), 343.
- Guidotti, R., Monreale, A., Giannotti, F., Pedreschi, D., Ruggieri, S., & Turini, F. (2019). Factual and counterfactual explanations for black box decision making. *IEEE Intelligent Systems*, 34(6), 14–23.
- Guidotti, R., Monreale, A., Ruggieri, S., Turini, F., Giannotti, F., & Pedreschi, D. (2019). A survey of methods for explaining black box models. *ACM Computing Surveys*, 51(5), 931–9342.
- Gulowaty, B., & Wozniak, M. (2021). Extracting interpretable decision tree ensemble from random forest. In *Proceedings of IJCNN 2021, Shenzhen, China, July 18–22, 2021* (pp. 1–8). IEEE.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference and prediction*. Berlin: Springer.
- Herbinger, J., Dandl, S., Ewald, F. K., Loibl, S., & Casalicchio, G. (2023). Leveraging model-based trees as interpretable surrogate models for model distillation. In *Proceedings of XI-ML workshop co-located with ECAI 2023, Poland, September 30–October 4, 2023* (pp. 232–249). Springer.
- Huang, X., & Marques-Silva, J. (2024). On the failings of shapley values for explainability. *International Journal of Approximate Reasoning* (p. 109112).
- Inglis, A., Parnell, A., & Hurley, C. B. (2022). Visualizing variable importance and variable interaction effects in machine learning models. *Journal of Computational and Graphical Statistics*, 31(3), 766–778.
- Keane, M. T., Kenny, E. M., Delaney, E., & Smyth, B. (2021). If only we had better counterfactual explanations: Five key deficits to rectify in the evaluation of counterfactual XAI techniques. In *Proceedings of the IJCAI 2021* (pp. 4466–4474).
- Liang, Y., Li, S., Yan, C., Li, M., & Jiang, C. (2021). Explaining the black-box model: A survey of local interpretation methods for deep neural networks. *Neurocomputing*, 419, 168–182.
- Li, L., Jamieson, K. G., DeSalvo, G., Rostamizadeh, A., & Talwalkar, A. (2017). Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18, 185:1–185:52.

- Lundberg, S. M., & Lee, S. (2017). A unified approach to interpreting model predictions. In *Proceedings of the NIPS 2017* (pp. 4765–4774).
- Lundberg, S. M., Erion, G. G., Chen, H., DeGrave, A. J., Prutkin, J. M., Nair, B., & Lee, S. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2(1), 56–67.
- Mann, H. B., & Whitney, D. R. (1947). On a test of whether one of two random variables is stochastically larger than the other. *Annals of Mathematical Statistics*, 18(1), 50–60.
- McElfresh, D. C., Khandagale, S., Valverde, J., C., V. P., Ramakrishnan, G., Goldblum, M., & White, C. (2023). When do neural nets outperform boosted trees on tabular data? In *Proceedings of the NeurIPS 2023*.
- Milani, S., Topin, N., Veloso, M., & Fang, F. (2024). Explainable reinforcement learning: A survey and comparative review. *ACM Computing Surveys*, 56(7), 168:1–168:36.
- Mitchell, R., Frank, E., & Holmes, G. (2022). Gputreeshap: Massively parallel exact calculation of SHAP scores for tree ensembles. *PeerJ Computer Science*, 8, Article e880.
- Molnar, C. (2025). *Interpretable machine learning* (3rd ed.). Munich, Germany: Self-published. Retrieved from <https://christophm.github.io/interpretable-ml-book>
- Pensa, R. G., Crombach, A., Peignier, S., & Rigotti, C. (2024). Combining shap-driven co-clustering and shallow decision trees to explain XGBoost. In *The proceedings of the DS 2024* (pp. 369–384).
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2018). Anchors: High-precision model-agnostic explanations. In *Proceedings of the AAAI/IAAI/EAAl 2018* (pp. 1527–1535).
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should I trust you?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD2016* (pp. 1135–1144).
- Robardet, C., & Feschet, F. (2001). Comparison of three objective functions for conceptual clustering. In *Proceedings of the PKDD 2001* (pp. 399–410).
- Sagi, O., & Rokach, L. (2020). Explainable decision forest: Transforming a decision forest into an interpretable tree. *Information Fusion*, 61, 124–138.
- Sagi, O., & Rokach, L. (2021). Approximating XGBoost with an interpretable decision tree. *Information Sciences*, 572, 522–542.
- Saleem, R., Yuan, B., Kurugollu, F., Anjum, A., & Liu, L. (2022). Explaining deep neural networks: A survey on the global interpretation methods. *Neurocomputing*, 513, 165–180.
- Shapley, L. S. (1953). *A value for n-person games. Contributions to the theory of games ii* (pp. 307–317). Princeton University Press.
- Shrikumar, A., Greenside, P., & Kundaje, A. (2017). Learning important features through propagating activation differences. In *Proceedings of the 34th international conference on machine learning* (Vol. 70, pp. 3145–3153).
- Staniak, M., & Biecek, P. (2018). Explanations of model predictions with live and breakdown packages. *The R Journal*, 10(2), 395–409.
- Strumbelj, E., & Kononenko, I. (2010). An efficient explanation of individual classifications using game theory. *Journal of Machine Learning Research*, 11, 1–18.
- Strumbelj, E., & Kononenko, I. (2014). Explaining prediction models and individual predictions with feature contributions. *Knowledge and Information Systems*, 41(3), 647–665.
- Vandewiele, G., Lannoye, K., Janssens, O., Ongenae, F., Turck, F. D., & Hoecke, S. V. (2017). A genetic algorithm for interpretable model extraction from decision tree ensembles. In *Proceedings of BDM workshop, co-located with PAKDD 2017* (pp. 104–115).
- Yang, J. (2021). Fast TreeSHAP: Accelerating SHAP value computation for trees. In *Proceedings of NeurIPS workshop on XAI approaches for debugging and diagnosis*.
- Yu, P., Bifet, A., Read, J., & Xu, C. (2022). Linear tree shap. In *NeurIPS Proceedings*.

Authors and Affiliations

Ruggero G. Pensa¹ · Anton Crombach^{2,3,4} · Sergio Peignier⁵ · Christophe Rigotti³

✉ Ruggero G. Pensa
ruggero.pensa@unito.it

Anton Crombach
anton.crombach@inria.fr

Sergio Peignier
sergio.peignier@insa-lyon.fr

Christophe Rigotti
christophe.rigotti@insa-lyon.fr

¹ Department of Computer Science, University of Turin, Turin, Italy

² Inria Centre de Lyon, Villeurbanne, France

³ INSA Lyon, CNRS, Universite Claude Bernard Lyon 1, LIRIS, UMR5205, Villeurbanne, France

⁴ INSA Lyon, CITI, UR3720, Villeurbanne, France

⁵ INSA Lyon, INRAE, BF2I, UMR0203, Villeurbanne, France